CHAPTER 03

Analysis of Linear Programming Models - Part 2

Solving LP with Python and CVXPY

TEXTBOOK	CHAPTER	WEEK
Modern Management Science, 9th ed.	Ch. 3	Week 4

Management Science · Fall 2026 · Kitae Kim, School of Business, Korea Aerospace University

Content prepared by the instructor; slide layout and formatting assisted by Claude (Anthropic).

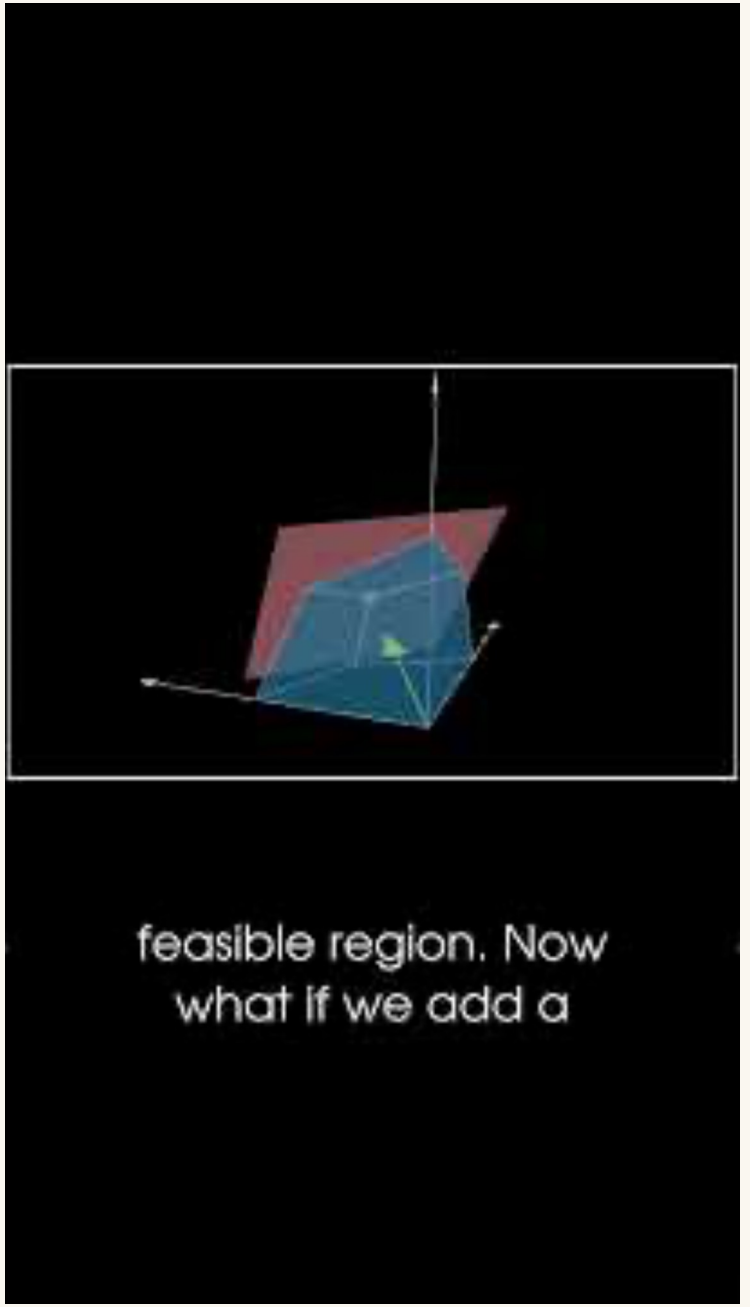
Last Week

Solving a LP with a graph

- Constraints on a graph
- The objective function on a graph
- Finding the optimal solution
- Slack variables and binding constraints

The simplex method

- The standard algorithm: the **Simplex Method**
  - Mathematically, the optimum is always at a corner point of the feasible region
  - Move from one corner to a better adjacent corner, **iteratively**, until no neighbouring corner is better
- We let a computer solve a large problem instead
  - If there are 3 decision variables, can you solve the problem by hand?
  - If there are hundreds of decision variables?



https://www.youtube.com/shorts/ZT3b2awWE1I

PART 2

Analysis with Software

From drawing the region to letting a solver find the corner

- Setting up the Colab environment
- Introduction to the CVXPY library
- Analyzing LP models with CVXPY (live demo): Examples 3-1 and 3-2

Setting up Google Colab

- **Colab (Colaboratory):** a free, cloud-based Python environment provided by Google
  - Notebooks are created and run directly from Google Drive
- **Quick start** (see Colab_Basics.pdf)
  1. Open Google Drive
  2. Install the Colab app (New → More → Connect more apps)
  3. Create a Jupyter notebook (.ipynb) file

All of today's code runs in Colab: no local Python installation needed



What is CVXPY?

- An **open-source, Python-embedded modeling language** for convex optimization problems
- You write the problem **the way the math reads**, not in the restrictive standard form a solver requires
- Linear programs are a special case of convex problems, so CVXPY handles everything in this chapter

Model in CVXPY, then hand the model to a solver of your choice

Source: https://www.cvxpy.org/



Many solvers, one interface

- CVXPY supports a wide range of solvers

  cvxpy.org/tutorial/solvers

- Which problems a solver can handle, and how well, differ by solver
  - Problem class: LP, QP, SOCP, MIP, ...
  - Performance: problem size, solution time, cost
- Widely used in industry (commercial)
  - GUROBI, MOSEK, CPLEX
- Widely used in research (free)
  - SCIPY, GLPK_MI (used in today's demo)

	LP	QP	SOCP	SDP	EXP	POW	MIP
CBC	X						X
CLARABEL	X	X	X	X	X	X	
COPT	X	X	X	X	X		X**
DAQP	X	X					
GLOP	X						
GLPK	X						
GLPK_MI	X						X
OSQP	X	X					
PIQP	X	X					
PROXQP	X	X					
QPALM	X	X					
PDLP	X						

Problem classes supported by each solver (cvxpy.org)

CVXPY: Input (Example 3-1)

EXAMPLE 3-1 · FULL CODE

```python
import cvxpy as cp

# (1) Decision variables
X1 = cp.Variable()
X2 = cp.Variable()

# (2) Objective function: maximize total profit
z = cp.Maximize(20 * X1 + 50 * X2)

# (3) Constraints
constraints = [
    1.2 * X1 + 4 * X2 <= 240,   # assembly hours
    0.5 * X1 + X2 <= 81,        # inspection hours
    X2 <= 40,                   # chip supply
    X1 >= 0, X2 >= 0            # non-negativity
]

# (4) Formulate the problem
prob = cp.Problem(z, constraints)

# (5) Solve it (verbose=True prints the solver log)
prob.solve(solver='GLPK_MI', verbose=True)

# (6) Print the results
print("Optimal X1:", X1.value)
print("Optimal X2:", X2.value)
print("Max profit Z = 20X1 + 50X2:", prob.value)
print("Status:", prob.status)
```

KEY CVXPY CALLS

Decision variables
`cp.Variable()`

Objective
`cp.Maximize(expr)` or `cp.Minimize(expr)`

Constraints
`a Python list: [c1, c2, c3, …]`

Problem
`prob = cp.Problem(objective, constraints)`

Solve
`prob.solve()`

choose a solver
`prob.solve(solver='NAME')`

show the solver log
`prob.solve(verbose=True)`

Results
`X1.value, prob.value, prob.status`

EXAMPLE 3-2

Example 3-2: a minimization problem

PROBLEM

The dietitian at the cafeteria of University A knows that, to stay healthy, each student must consume 66 g of protein and 9 mg of iron per day. These nutrients come from meat and vegetables: 1 kg of meat contains 300 g of protein and 30 mg of iron, and 1 kg of vegetables contains 20 g of protein and 10 mg of iron. The cafeteria buys meat at ₩6,000 per kg and vegetables at ₩1,000 per kg. The dietitian wants a purchasing plan that keeps students healthy at the lowest possible food cost.

FORMULATION

- **Decision variables**
- **Objective function**
- **Constraints**

Set up the model first: variables, objective, constraints

Example 3-2: a minimization problem

PROBLEM

The dietitian at the cafeteria of University A knows that, to stay healthy, each student must consume 66 g of protein and 9 mg of iron per day. These nutrients come from meat and vegetables: 1 kg of meat contains 300 g of protein and 30 mg of iron, and 1 kg of vegetables contains 20 g of protein and 10 mg of iron. The cafeteria buys meat at ₩6,000 per kg and vegetables at ₩1,000 per kg. The dietitian wants a purchasing plan that keeps students healthy at the lowest possible food cost.

FORMULATION

- **Decision variables**
  - X_1 = kg of meat bought per person per day
  - X_2 = kg of vegetables bought per person per day
- **Objective function**
  - **Minimize** $Z = 6000X_1 + 1000X_2$ (₩)
- **Constraints**
  - Protein: $300X_1 + 20X_2 \geq 66$ (g)
  - Iron: $30X_1 + 10X_2 \geq 9$ (mg)
  - Non-negativity: $X_1, X_2 \geq 0$

Example 3-2: from model to code

FORMULATION

- **Decision variables**
  - X_1 = kg of meat per person per day
  - X_2 = kg of vegetables per person per day
- **Objective function**
  - **Minimize** $Z = 6000X_1 + 1000X_2$
- **Constraints**
  - $300X_1 + 20X_2 \geq 66$ (protein, g)
  - $30X_1 + 10X_2 \geq 9$ (iron, mg)
  - $X_1, X_2 \geq 0$

INPUT

```
import cvxpy as cp

# 1. Decision variables
x1 = cp.Variable()
x2 = cp.Variable()

# 2. Objective: minimize total food cost
z = 6000 * x1 + 1000 * x2
objective = cp.Minimize(z)

# 3. Constraints: minimum nutrient requirements
constraints = [
    300 * x1 + 20 * x2 >= 66,   # protein >= 66 g
    30 * x1 + 10 * x2 >= 9,     # iron >= 9 mg
    x1 >= 0,                    # non-negativity
    x2 >= 0
]

# 4. Formulate and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# 5. Print the results
print("Status:", problem.status)
print("Minimum food cost:", problem.value, "won")
print("Meat (X1):", x1.value, "kg")
print("Vegetables (X2):", x2.value, "kg")
```

Example 3-2: input and output

INPUT

```
import cvxpy as cp

# 1. Decision variables
x1 = cp.Variable()
x2 = cp.Variable()

# 2. Objective: minimize total food cost
z = 6000 * x1 + 1000 * x2
objective = cp.Minimize(z)

# 3. Constraints: minimum nutrient requirements
constraints = [
    300 * x1 + 20 * x2 >= 66,    # protein >= 66 g
    30 * x1 + 10 * x2 >= 9,      # iron >= 9 mg
    x1 >= 0,                     # non-negativity
    x2 >= 0
]

# 4. Formulate and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# 5. Print the results
print("Status:", problem.status)
print("Minimum food cost:", problem.value, "won")
print("Meat (X1):", x1.value, "kg")
print("Vegetables (X2):", x2.value, "kg")
```

OUTPUT

```
Status: optimal
Minimum food cost: 1499.999996424232 won
Meat (X1): 0.19999999972075516 kg
Vegetables (X2): 0.29999999809970107 kg
```

- Status is optimal: the solver found the true minimum
- Buy 0.2 kg of meat and 0.3 kg of vegetables per person per day
- The long decimals (0.1999…) are floating-point noise

Minimum daily food cost: ₩1,500 per student

Summary

- Analyzing optimization problems with software
  - The basic algorithm: the simplex method
  - A Python-based modeling package: CVXPY
  - Many solvers exist, so choose one that suits your problem
  - Solvers differ in the problem classes and sizes they can handle
  - Solvers also search for the optimum differently, so their performance differs

Model once in CVXPY; switch solvers with a single argument

CVXPY IN SIX STEPS

```
1  Decision variables
x1 = cp.Variable()

2  Objective
z = cp.Maximize(...)  or  cp.Minimize(...)

3  Constraints
constraints = [c1, ..., cn]

4  Problem
prob = cp.Problem(z, constraints)

5  Solve
prob.solve()

6  Results
x1.value,  prob.value,  prob.status
```